LAB 06_REPORT

Inheritance OOP

NOVEMBER 14, 2026

NAME: SIEM KHATTYAVATEY
Student ID: p20250056
GIT URL: https://github.com/zikkyiscool/PDI.git

Practice 1:

Use case: A transport company wants a common structure for different vehicles.

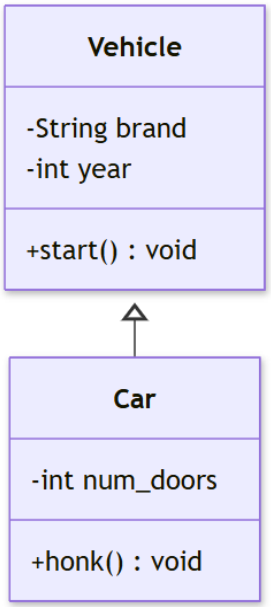
Goal: Understand simple inheritance.

Instructions:

- Create a base class Vehicle with:
  - properties: brand, year
  - method: start()
- Create a derived class Car that inherits from Vehicle.
- Add car-specific property: num_doors.
- Add method: honk().

Task: Create objects of both classes and call their methods to show that Car has access to Vehicle methods.

Class diagram



Implement:

Vehicle.java: This file defines the common properties and behaviors shared by vehicles.

```java
package Exercise_1;
public class Vehicle{
    String brand;
    int year;

    void start(){
        System.out.println("The vehicle is start! ");
    }
}
```

Car.java: This file defines the Car class, which inherits from Vehicle.

```java
package Exercise_1;

You, 18 minutes ago | author (You)
public class Car extends Vehicle{
    int num_doors;

    void honk(){
        System.out.println("Honk Honk Honk");
    }
}
```

Main.java: This file runs the program. It creates objects and calls their methods.

```java
package Exercise_1;

You, 20 minutes ago | author (You)
public class Main {
    Run | Debug
    public static void main(String[]args){
        Vehicle v1 = new Vehicle();
        v1.brand = "Mazdar";
        v1.year = 2026;

        Car c1 = new Car();
        c1.brand = "Honda";
        c1.year = 2025;
        c1.num_doors = 4;

        c1.start();
        c1.honk();

        System.out.println("Brand: " + c1.brand);
        System.out.println("Year: " + c1.year);
        System.out.println("Door: " + c1.num_doors);
    }
}
```

Output:

```
The vehicle is start!
Honk Honk Honk
Brand: Honda
Year: 2025
Door: 4
PS D:\PDI>
```

Practice 2: Method Overriding — "Employee & Manager"

Use case: A company payroll system needs different bonus calculations.

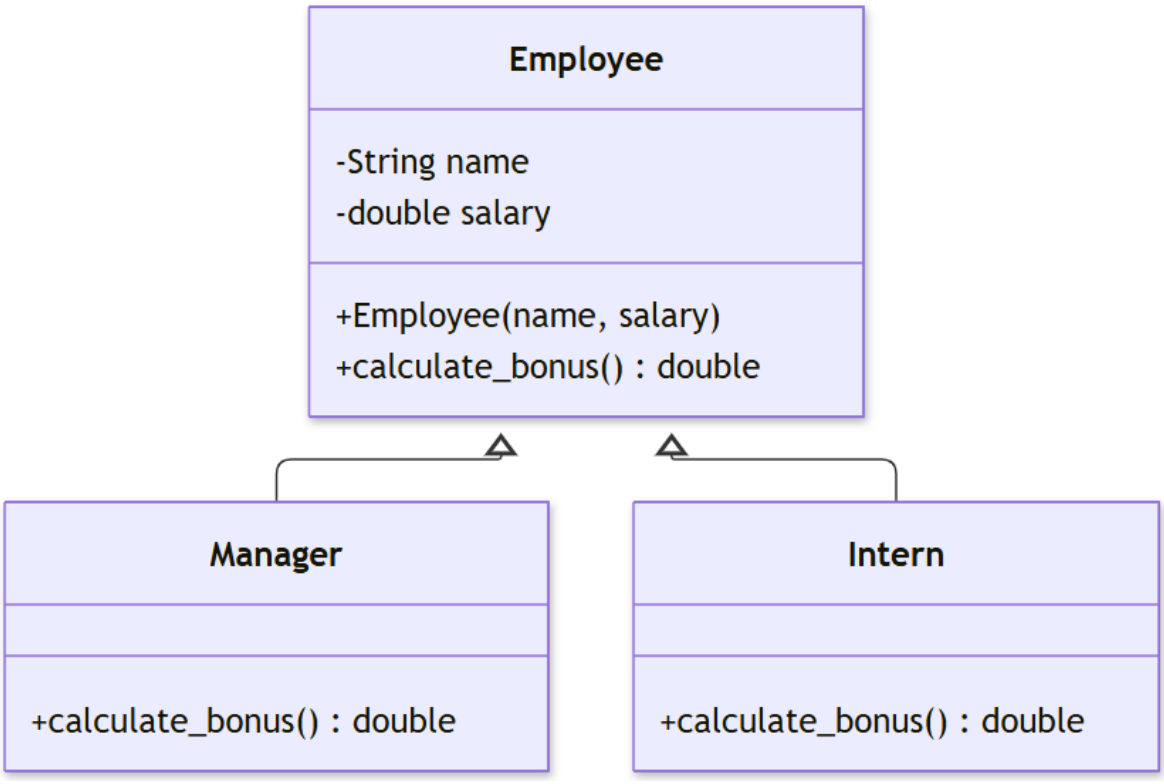
Goal: Use overridden methods.

Instructions:

- Base class Employee with:
  - name, salary
  - method: calculate_bonus() → returns 10% of salary
- Create a derived class Car that inherits from Vehicle.
- Add car-specific property: num_doors.
- Add method: honk().

Task: Print the bonus for each type to show polymorphism.

Class diagram



Implement:

Employee.java: This file defines the base Employee class with name and salary attributes. It also contains the calculate_bonus() method, which calculates a default bonus of 10% of the employee's salary.

```java
package Exercise_2;

You, 55 minutes ago | author (You)
public class Employee {
    String name;
    double salary;

    Employee(String name, double salary){
        this.name = name;
        this.salary = salary;
    }

    public double calculate_bonus(){
        return salary * 0.1 ;
    }
}
```

Manager.java: This file defines the Manager class, which inherits from Employee. It overrides the calculate_bonus() method to calculate a 20% bonus instead of the default 10%.

```java
package Exercise_2;

You, 1 hour ago | author (You)
public class Manager extends Employee{
    Manager(String name, double salary) {
        super(name, salary);
    }

    @Override
    public double calculate_bonus(){
        return salary * 0.2;
    }

}
```

Intern.java: This file defines the Intern class, which inherits from Employee. It overrides the calculate_bonus() method to calculate a 5% bonus.

```java
package Exercise_2;

You, 1 hour ago | author (You)
public class Intern extends Employee {
    Intern(String name, double salary) {
        super(name, salary);
    }

    @Override
    public double calculate_bonus(){
        return salary * 0.05;
    }
}
```

Main.java: This file runs the program by creating Employee, Manager, and Intern objects. It uses a loop to call calculate_bonus() for each object, demonstrating polymorphism because the same method produces different results depending on the object type.

```java
package Exercise_2;

You, 2 hours ago | author (You)
public class Main {
    Run | Debug
    public static void main(String[]args){

        Employee e1 = new Employee(name: "Dara", salary: 550);
        Employee e2 = new Manager(name: "Sok", salary: 1000);
        Employee e3 = new Intern(name: "Roth",salary: 200);

        System.out.println("=======Employee List=======");
        System.out.println("Name: " + e1.name);
        System.out.println("Position: Normal employee");
        System.out.println("Salary: " + e1.salary);
        System.out.println("Bonus: " + e1.calculate_bonus());
        System.out.println("---------------------------");

        System.out.println("Name: " + e2.name);
        System.out.println("Position: Manager");
        System.out.println("Salary: " + e2.salary);
        System.out.println("Bonus: " + e2.calculate_bonus());
        System.out.println("---------------------------");

        System.out.println("Name: " + e3.name);
        System.out.println("Position: Intern");
        System.out.println("Salary: " + e3.salary);
        System.out.println("Bonus: " + e3.calculate_bonus());
        System.out.println("---------------------------");
    }

}
```

Output:

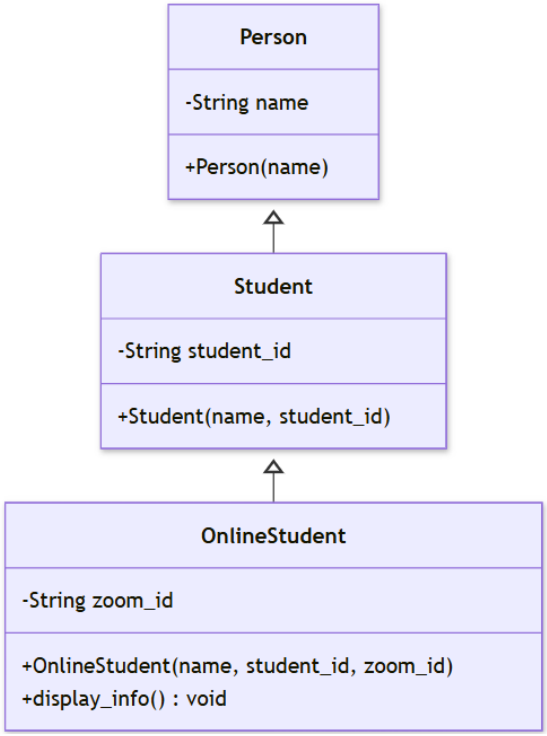
```
=======Employee List=======
Name: Dara
Position: Normal employee
Salary: 550.0
Bonus: 55.0
---------------------------
Name: Sok
Position: Manager
Salary: 1000.0
Bonus: 200.0
---------------------------
Name: Roth
Position: Intern
Salary: 200.0
Bonus: 10.0
---------------------------
PS D:\PDI>
```

Practice 3: Multi-Level Inheritance — "Person → Student → OnlineStudent"
Use case: A university system has different student types.
Goal: Build a multi-level class chain.
Instructions:

- Base: Person → name
- Derived: Student → student_id
- Derived: OnlineStudent → zoom_id

Task: Instantiate OnlineStudent and confirm it inherits all data from both parent classes.

Class diagram



Implement:
Person.java: This file defines the Person class with a name attribute. It provides a constructor to initialize the person's name and a getter method to access it.

```java
package Exercise_3;

...
public class Person {
    String name;

    Person(String name){
        this.name = name;
    }
}
```

Student.java: This file defines the Student class, which inherits from Person. It adds a student_id attribute and uses super(name) to initialize the name inherited from the parent class.

```java
package Exercise_3;

You, 2 hours ago | author (You)
public class Student extends Person {
    String student_id;

    Student(String name, String student_id){
        super(name) ;
        this.student_id = student_id;
    }
}
```

OnlineStudent.java: This file defines the OnlineStudent class, which inherits from Student. It adds a zoom_id attribute and uses super(name, student_id) to initialize the information inherited from both parent classes. The display_info() method displays all three attributes.

```java
package Exercise_3;

You, 2 hours ago | author (You)
public class OnlineStudent extends Student {
    String zoom_id;

    OnlineStudent( String name, String student_id, String  zoom_id){
        super(name, student_id);
        this.zoom_id = zoom_id;
    }

    void display(){
        System.out.println("Name: " + name);
        System.out.println("Student ID: " + student_id);
        System.out.println("Zoom ID: " + zoom_id);
    }
}
```

Main.java: This file creates an OnlineStudent object and calls display_info() to demonstrate that the object can access data inherited from both Person and Student, as well as its own zoom_id.

```java
package Exercise_3;

You, 2 hours ago | author (You)
public class Main {
    Run | Debug
    public static void main(String[]args){

    OnlineStudent s1 = new OnlineStudent(name: "Rotha",student_id: "p20250012",zoom_id: "p27748");

    System.out.println("Name: " + s1.name);
    System.out.println("Student ID: " + s1.student_id);
    System.out.println("Zoom ID: " + s1.zoom_id);
    }
}
```

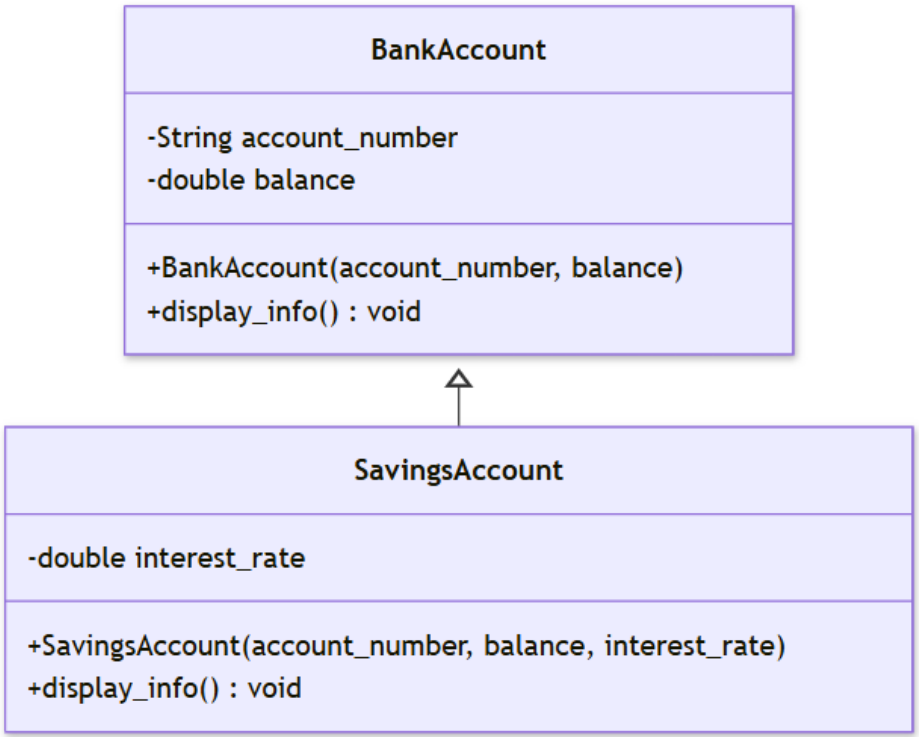
Output:

```
Name: Rotha
Student ID: p20250012
Zoom ID: p27748
PS D:\PDI\Lab06>
```

Practice 4: Calling Parent Constructor — "Bank Account Types"
Use case: A bank system with multiple account categories.
Goal: Learn super() or base-constructor calls.
Instructions:

- Base: BankAccount
  - constructor: accepts account_number, balance
- Derived: SavingsAccount
  - constructor: also takes interest_rate
  - must call the base constructor

Task: Show correct initialization of subclass properties.
Class diagram
This exercise demonstrates how a child class uses super() to call the constructor of its parent class.



Implement:

BankAccount.java: This file defines the parent BankAccount class with account_number and balance attributes. Its constructor initializes these two properties when a bank account object is created.

```java
package Exercise_4;

You, 3 hours ago | author (You)
public class BankAccount {
    String account_number;
    double balance;

    public BankAccount(String account_number, double balance){
        this.account_number = account_number;
        this.balance = balance;
    }
}
```

SavingsAccount.java: This file defines SavingsAccount, which inherits from BankAccount. It adds an interest_rate attribute and uses super(account_number, balance) to call the parent constructor and initialize the inherited properties.

```java
package Exercise_4;

You, 3 hours ago | author (You)
public class SavingsAccount extends BankAccount{
    double interest_rate;

    public SavingsAccount(String account_number, double balance, double interest_rate){
        super(account_number, balance);
        this.interest_rate = interest_rate;
    }

}
```

Main.java: This file creates a SavingsAccount object with an account number, balance, and interest rate. It then calls display_info() to display all the initialized information.

```java
package Exercise_4;

You, 3 hours ago | author (You)
public class Main {
    Run | Debug
    public static void main (String[]arge){
        SavingsAccount a1 = new SavingsAccount(
            account_number: "123456789", balance: 1900,interest_rate: 5);

        System.out.println("Account number: " + a1.account_number);
        System.out.println("Balance: " + a1.balance);
        System.out.println("Interest: " + a1.interest_rate);
    }

}
```

Output:

```
Account number: 123456789
Balance: 1900.0
Interest: 5.0
PS D:\PDI\Lab06>
```

Practice 5: Using Protected Members — "Library Resources"

Use case: A library manages different resource types: books, magazines, journals.

Goal: Practice protected attributes and subclass access.

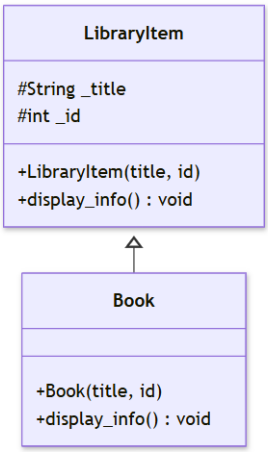
Instructions:

- Base class LibraryItem
  - protected: _title, _id
  - method: display_info()
- Derived class Book
  - method: display_info() → formats book output differently

Task: Demonstrate how _title is accessible in the subclass but not outside.

Class diagram

This exercise demonstrates protected attributes, which allow a child class to access its parent's attributes directly.



Implement:

LibraryItem.java: This file defines the parent LibraryItem class with two protected attributes, _title and _id. The constructor initializes these attributes, while display_info() displays the library item's information.

```java
package Exercise_5;

You, 3 hours ago | author (You)
public class LibraryItem {
    protected String _title;
    protected int _id;

    public LibraryItem(String title, int id) {
        this._title = title;
        this._id = id;
    }

    public void display_info() {
        System.out.println("Title: " + _title);
        System.out.println("ID: " + _id);
    }
}
```

Book.java: This file defines the Book class, which inherits from LibraryItem. It uses super() to initialize the parent's attributes and overrides display_info() to format the book information differently. Because _title and _id are protected, the Book subclass can access them directly.

```java
package Exercise_5;

You, 3 hours ago | author (You)
public class Book extends LibraryItem {

    public Book(String title, int id) {
        super(title, id);
    }

    @Override
    public void display_info() {
        System.out.println("=== Book Information ===");
        System.out.println("Book Title: " + _title);
        System.out.println("Book ID: " + _id);
    }
}
```

Main.java: This file creates a Book object and calls display_info() to display its information. It does not directly access _title or _id because Main is outside the library package.

```java
package Exercise_5;

You, 3 hours ago | author (You)
public class Main {
    Run | Debug
    public static void main(String[] args) {

        Book b1 = new Book(title: "Java Programming", id: 101);

        b1.display_info();

        // Cannot directly access protected members
        // from an unrelated class outside the package.
        // System.out.println(b1._title); // Error
    }
}
```

Output:

```
=== Book Information ===
Book Title: Java Programming
Book ID: 101
PS D:\PDI\Lab06>
```

Practice 6: Hierarchical Inheritance — "Shape → Circle, Rectangle, Triangle"

Use case: A drawing application must compute areas of different shapes.

Goal: Use one parent with multiple children.

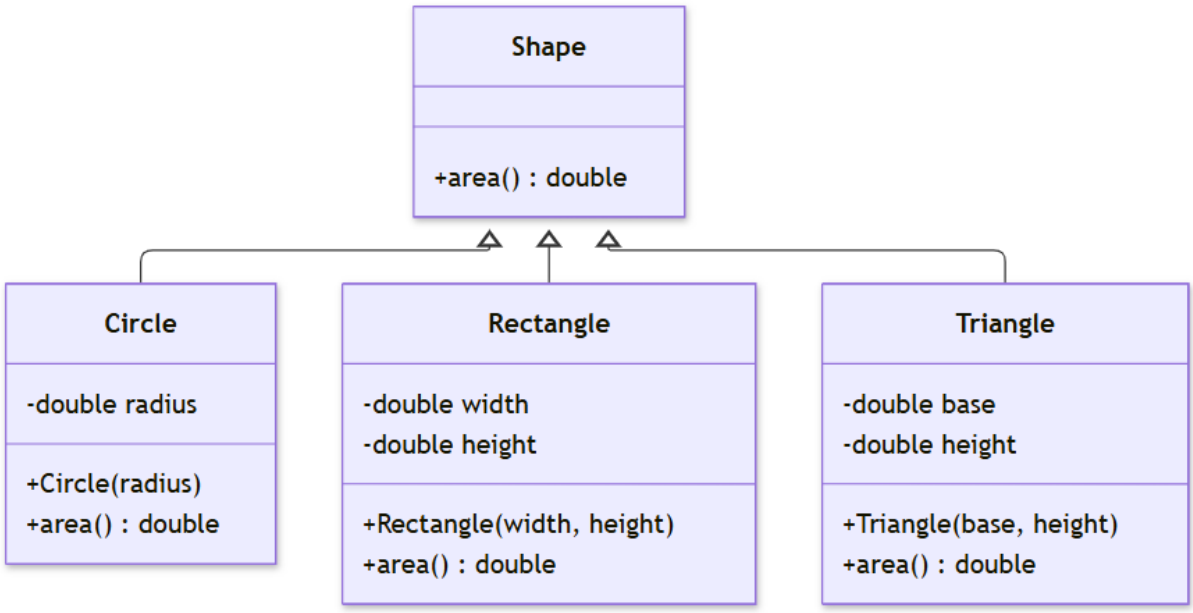
Instructions:

- Base: Shape o method: area() → returns 0 (default)
- Child: Circle
- Child: Rectangle
- Child: Triangle
- Each child overrides area().

Task: Store different shape objects in a list and print their areas.

Class diagram

This exercise demonstrates hierarchical inheritance, where multiple child classes inherit from one parent class.



Implement:

Shape.java: This file defines the parent Shape class with a default area() method that returns 0. The child classes inherit this method and override it to calculate their own areas.

```java
package Exercise_6;

You, 4 hours ago | author (You)
public class Shape {

    public double area(){
        return 0;
    }
}
```

Circle.java: This file defines the Circle class, which inherits from Shape. It adds a radius attribute and overrides area() to calculate the circle's area $\pi \times radius \times radius$.

```java
package Exercise_6;

You, 4 hours ago | author (You)
public class Circle extends Shape {
    double radius;
    public Circle(double radius){
        this.radius = radius;
    }

    @Override
    public double area(){
        return 3.14 * radius * radius ;
    }
}
```

Rectangle.java: This file defines the Rectangle class, which inherits from Shape. It adds width and height attributes and overrides area() to calculate the rectangle's area by multiplying its width by its height.

```java
package Exercise_6;

You, 4 hours ago | author (You)
public class Rectangle extends Shape {
    double length;
    double width;
    public Rectangle(double length, double width){
        this.length = length;
        this.width = width;
    }

    @Override
    public double area(){
        return length * length ;
    }
}
```

Triangle.java: This file defines the Triangle class, which inherits from Shape. It adds base and height attributes and overrides area() to calculate the triangle's area using half of its base multiplied by its height.

```java
package Exercise_6;

...
public class Triangle extends Shape {
    double height;
    double base;
    public Triangle(double height, double base){
        this.height = height;
        this.base = base;
    }

    @Override
    public double area(){
        return (height * base)/2 ;
    }
}
```

Main.java: This file creates an ArrayList that stores different shape objects. It adds a Circle, Rectangle, and Triangle, then uses a for-each loop to call area() on each object. Java automatically executes the overridden method belonging to each actual object.

```java
package Exercise_6;

import java.util.ArrayList;

You, 4 hours ago | author (You)
public class Main {
    Run | Debug
    public static void main(String[]args){
        ArrayList<Shape> shapes = new ArrayList<>();

        shapes.add(new Circle(radius: 5));
        shapes.add(new Rectangle(length: 4, width: 6));
        shapes.add(new Triangle(height: 3, base: 8));

        for (Shape s : shapes) {
            System.out.println("Area: " + s.area());
        }
    }
}
```

Output:

```
Area: 78.5
Area: 16.0
Area: 12.0
PS D:\PDI\Lab06>
```

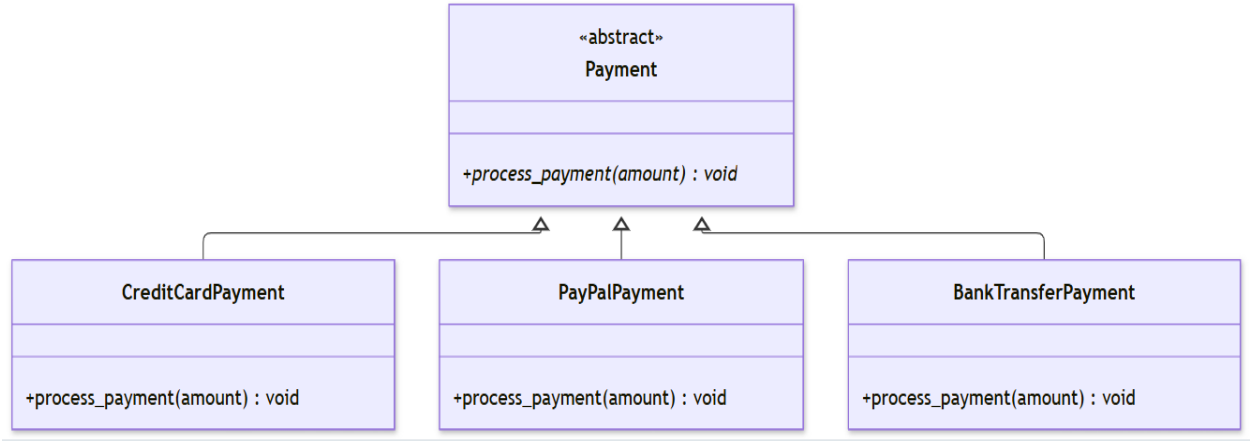
Practice 7: Abstract Classes — "Payment Methods"
Use case: E-commerce needs different payment implementations.
Goal: Use abstract classes and enforce method overriding.
Instructions:

- Abstract class: Payment
  - abstract method: process_payment(amount)
- Derived:
  - CreditCardPayment
  - PayPalPayment
  - BankTransferPayment

Task: Invoke each payment method and show different behavior.

Class diagram

This exercise demonstrates abstraction, where an abstract parent class defines a method that every child class must implement.



Implement:

Payment.java: This file defines the abstract parent class Payment. It declares an abstract method process_payment() without providing an implementation. Every child class must override this method to define its own payment behavior.

```java
package Exercise_7;

You, 4 hours ago | author (You)
abstract class Payment {
    abstract void process_payment(double amount);
}
```

CreditCardPayment.java: This file defines CreditCardPayment, which inherits from Payment. It overrides process_payment() to display a message indicating that the payment is being processed using a credit card.

```java
package Exercise_7;

You, 4 hours ago | author (You)
class CreditCardPayment extends Payment {

    @Override
    void process_payment(double amount) {
        System.out.println("Processing credit card payment: $" + amount);
    }
}
```

PayPalPayment.java: This file defines PayPalPayment, which inherits from Payment. It overrides process_payment() to display a message indicating that the payment is being processed through PayPal.

```java
package Exercise_7;

You, 4 hours ago | author (You)
class PayPalPayment extends Payment {

    @Override
    void process_payment(double amount) {
        System.out.println("Processing PayPal payment: $" + amount);
    }
}
```

BankTransferPayment.java: This file defines BankTransferPayment, which inherits from Payment. It overrides process_payment() to display a message indicating that the payment is being processed through a bank transfer.

```java
package Exercise_7;

You, 4 hours ago | author (You)
class BankTransferPayment extends Payment {

    @Override
    void process_payment(double amount) {
        System.out.println("Processing bank transfer payment: $" + amount);
    }
}
```

Main.java: This file creates three different payment objects using the Payment reference type. It calls process_payment() for each object to demonstrate polymorphism, where the same method call produces different behavior depending on the payment type.

```java
package Exercise_7;

You, 4 hours ago | author (You)
public class Main {

    Run | Debug
    public static void main(String[] args) {

        Payment p1 = new CreditCardPayment();
        Payment p2 = new PayPalPayment();
        Payment p3 = new BankTransferPayment();

        p1.process_payment(amount: 100.00);
        p2.process_payment(amount: 250.00);
        p3.process_payment(amount: 500.00);

    }
}
```

Output:

```
Processing credit card payment: $100.0
Processing PayPal payment: $250.0
Processing bank transfer payment: $500.0
PS D:\PDI\Lab06>
```

Practice 8: Inheritance + Polymorphism + Data Structures — "E-commerce Product Catalog"

Use case: Online store with several product types.

Goal: Work with lists of parent-class types containing child objects.

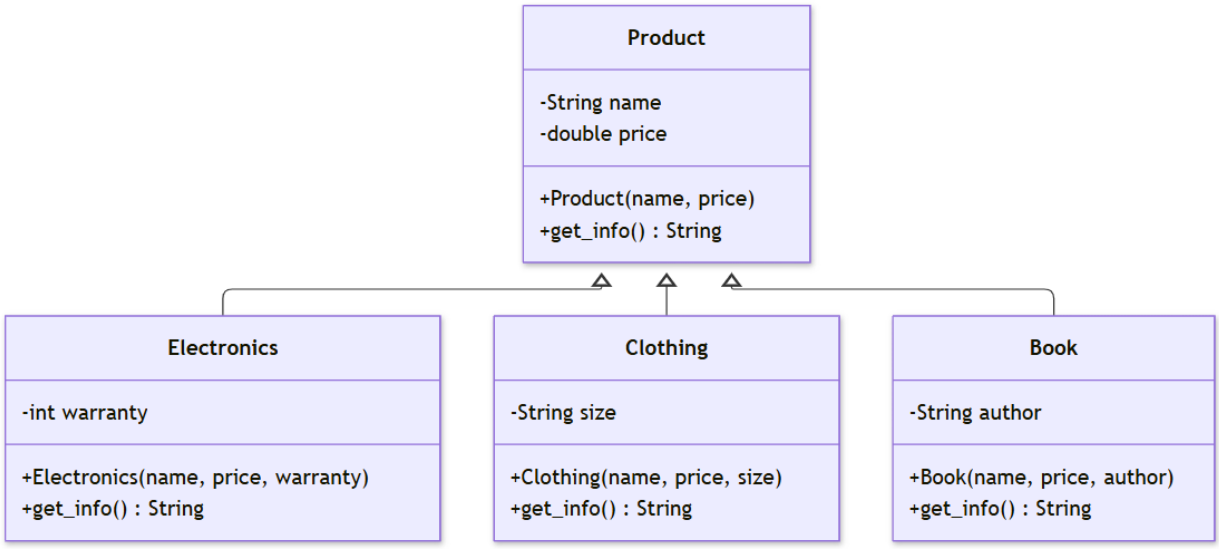
Instructions:

- Base class Product: name, price
- Derived:
  - Electronics → warranty
  - Clothing → size
  - Book → author
- Add method get_info() in each class (overridden).

Task: Create a product list containing all types and print info using polymorphism.

Class diagram

This exercise demonstrates inheritance, method overriding, polymorphism, and ArrayList. One parent class has three child classes, each with its own implementation of get_info().



Implement:

Product.java: This file defines the parent Product class with name and price attributes. It provides a constructor to initialize product information, getter methods to access the attributes, and a default get_info() method that returns product details.

```java
package Exercise_8;

You, 5 hours ago | author (You)
class Product {
    protected String name;
    protected double price;

    public Product(String name, double price) {
        this.name = name;
        this.price = price;
    }

    public void get_info() {
            System.out.println("Product: " + name);
            System.out.println("Price: $" + price);
    }
}
```

Electronics.java: This file defines the Electronics class, which inherits from Product. It adds a warranty attribute and overrides get_info() to display the product name, price, and warranty period.

```java
package Exercise_8;

You, 4 hours ago | author (You)
class Electronics extends Product {
    private int warranty;

    public Electronics(String name, double price, int warranty) {
        super(name, price);
        this.warranty = warranty;
    }

    @Override
    public void get_info() {
        System.out.println("Electronics: " + name);
        System.out.println("Price: $" + price);
        System.out.println("Warranty: " + warranty + " years");
    }
}
```

Clothing.java: This file defines the Clothing class, which inherits from Product. It adds a size attribute and overrides get_info() to display the clothing name, price, and size.

```java
package Exercise_8;

You, 4 hours ago | author (You)
class Clothing extends Product {
    private String size;

    public Clothing(String name, double price, String size) {
        super(name, price);
        this.size = size;
    }

    @Override
    public void get_info() {
        System.out.println("Clothing: " + name);
        System.out.println("Price: $" + price);
        System.out.println("Size: " + size);
    }
}
```

Book.java: This file defines the Book class, which inherits from Product. It adds an author attribute and overrides get_info() to display the book name, price, and author's name.

```java
package Exercise_8;

...
class Book extends Product {
    private String author;

    public Book(String name, double price, String author) {
        super(name, price);
        this.author = author;
    }

    @Override
    public void get_info() {
        System.out.println("Book: " + name);
        System.out.println("Price: $" + price);
        System.out.println("Author: " + author);
    }
}
```

Main.java: This file creates an ArrayList<Product> that stores Electronics, Clothing, and Book objects. It uses a for-each loop to call get_info() on each product. Java automatically executes the overridden method belonging to the actual object, demonstrating polymorphism.

```java
package Exercise_8;

import java.util.ArrayList;

You, 4 hours ago | author (You)
public class Main {
    Run | Debug
    public static void main(String[] args) {

        ArrayList<Product> products = new ArrayList<>();

        products.add(new Electronics(name: "Laptop", price: 850, warranty: 2));
        products.add(new Clothing(name: "T-Shirt", price: 15, size: "L"));
        products.add(new Book(name: "Java Programming", price: 30, author: "James Gosling"));

        for (Product p : products) {
            p.get_info();
            System.out.println("----------------------");
        }
    }
}
```

Output:

```
Electronics: Laptop
Price: $850.0
Warranty: 2 years
----------------------
Clothing: T-Shirt
Price: $15.0
Size: L
----------------------
Book: Java Programming
Price: $30.0
Author: James Gosling
----------------------
```

Practice 9: Advanced Real-World Project — "Transportation System with Pricing Rules"

Use case: A city transportation app calculates fares for different transport modes.

Goal: Use inheritance, overriding, polymorphism, abstract classes, and real logic.

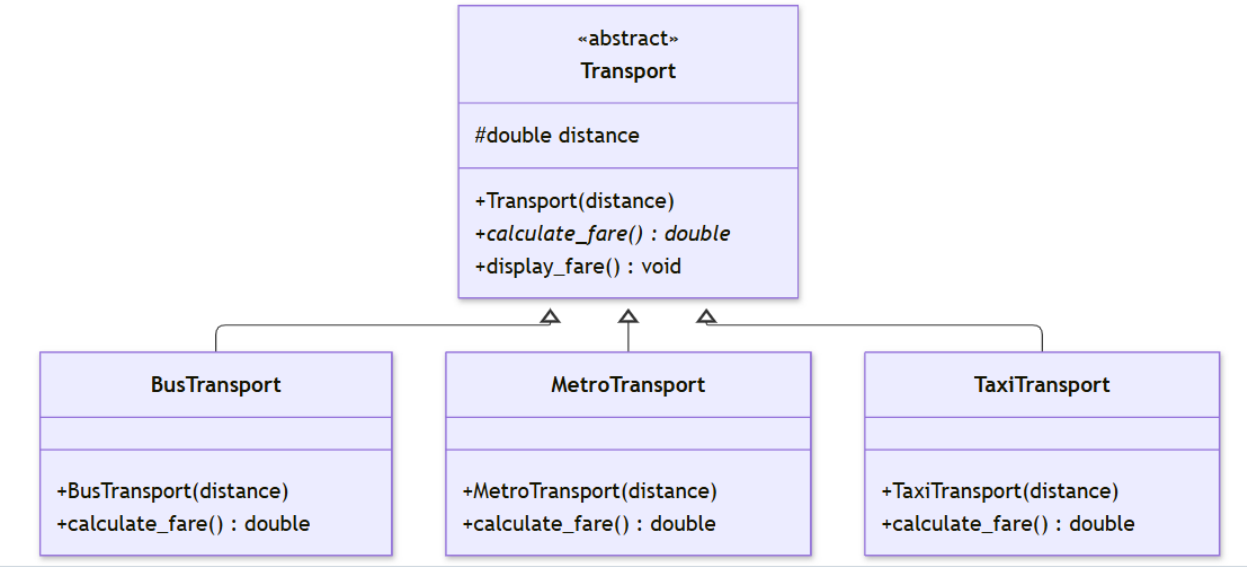
Instructions:

- Abstract class: Transport
  - property: distance
  - abstract method: calculate_fare()
- Derived:
  - BusTransport → fare = distance × 1.5
  - MetroTransport → fare = base 10 + (distance × 2)
  - TaxiTransport → starting fee 20 + distance × 5
- Add method: display_fare()

Task: Create a list of multiple transport types with various distances and show each fare computed using the overridden methods.

Class diagram

This exercise combines abstract classes, inheritance, method overriding, polymorphism, and ArrayList.



Implement:

Transport.java: This file defines the abstract parent class Transport with a protected distance attribute. It declares the abstract method calculate_fare(), which must be implemented by child classes. It also provides display_fare() to display the distance and calculated fare.

```java
package Exercise_9;

You, 5 hours ago | author (You)
public abstract class Transport {

    protected double distance;

    public Transport(double distance) {
        this.distance = distance;
    }

    public abstract double calculate_fare();

    public void display_fare() {
        System.out.println("Distance: " + distance + " km");
        System.out.println("Fare: $" + calculate_fare());
    }
}
```

BusTransport.java: This file defines the BusTransport class, which inherits from Transport. It uses super(distance) to initialize the parent's distance attribute and overrides calculate_fare() to calculate the bus fare at $1.50 per kilometer.

```java
package Exercise_9;

You, 5 hours ago | author (You)
public class BusTransport extends Transport {

    public BusTransport(double distance) {
        super(distance);
    }

    @Override
    public double calculate_fare() {
        return distance * 1.5;
    }
}
```

MetroTransport.java: This file defines the MetroTransport class, which inherits from Transport. It overrides calculate_fare() to calculate the metro fare using a base fare of $10 plus $2 per kilometer.

```java
package Exercise_9;

You, 5 hours ago | author (You)
public class MetroTransport extends Transport {

    public MetroTransport(double distance) {
        super(distance);
    }

    @Override
    public double calculate_fare() {
        return 10 + (distance * 2);
    }
}
```

TaxiTransport.java: This file defines the TaxiTransport class, which inherits from Transport. It overrides calculate_fare() to calculate the taxi fare using a starting fee of $20 plus $5 per kilometer.

```java
package Exercise_9;

You, 5 hours ago | author (You)
public class TaxiTransport extends Transport {

    public TaxiTransport(double distance) {
        super(distance);
    }

    @Override
    public double calculate_fare() {
        return 20 + (distance * 5);
    }
}
```

Main.java: This file creates an ArrayList<Transport> to store BusTransport, MetroTransport, and TaxiTransport objects. It uses a for-each loop to call display_fare() for each object. Inside that method, Java automatically calls the correct overridden calculate_fare() based on the actual transport type.

```java
package Exercise_9;

import java.util.ArrayList;

You, 5 hours ago | author (You)
public class Main {

    Run | Debug
    public static void main(String[] args) {

        ArrayList<Transport> transports = new ArrayList<>();

        transports.add(new BusTransport(distance: 10));
        transports.add(new MetroTransport(distance: 8));
        transports.add(new TaxiTransport(distance: 5));

        transports.add(new BusTransport(distance: 20));
        transports.add(new MetroTransport(distance: 12));
        transports.add(new TaxiTransport(distance: 10));

        for (Transport t : transports) {

            System.out.println("Transport Type: "
                    + t.getClass().getSimpleName());

            t.display_fare();

            System.out.println("----------------------");
        }
    }
}
```

Output:

```
Transport Type: BusTransport
Distance: 10.0 km
Fare: $15.0
----------------------
Transport Type: MetroTransport
Distance: 8.0 km
Fare: $26.0
----------------------
Transport Type: TaxiTransport
Distance: 5.0 km
Fare: $45.0
----------------------
```

```
Transport Type: BusTransport
Distance: 20.0 km
Fare: $30.0
----------------------
Transport Type: MetroTransport
Distance: 12.0 km
Fare: $34.0
----------------------
Transport Type: TaxiTransport
Distance: 10.0 km
Fare: $70.0
----------------------
PS D:\PDI\Lab06>
```

THE END